FICHE DE SYNTHÈSE

Méthode du simplexe de Nelder–Mead

1 Présentation générale du projet

L'objectif du projet est d'implémenter en C++ la méthode de Nelder–Mead afin de rechercher le minimum d'une fonction sans calcul explicite du gradient. Le projet met aussi en pratique plusieurs notions de programmation orientée objet et permet de visualiser l'évolution des simplexes au cours des itérations. Une extension traite enfin une recherche de minimum sous contrainte de type hyperplan.

2 Une classe `Simplexe`

La classe `Simplexe` représente un simplexe par sa dimension et ses sommets. Mathématiquement, un simplexe de dimension n possède exactement $n+1$ sommets : un segment en dimension 1, un triangle en dimension 2 et un tétraèdre en dimension 3. Chaque sommet est un vecteur de $\mathbb{R}^n$:

$$X_i = \begin{pmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{in} \end{pmatrix}.$$

L'invariant fondamental imposé par le constructeur est :

$$\boxed{\text{nombre de sommets} = \text{dimension} + 1}.$$

Le programme vérifie également que chaque sommet possède exactement n coordonnées. La classe fournit des méthodes pour obtenir la dimension, le nombre de sommets, lire un sommet et remplacer un sommet.

3 L'algorithme de recherche du minimum

Le coeur du projet est la méthode de Nelder–Mead. Le problème considéré est

$$\min_{x \in \mathbb{R}^n} f(x).$$

La méthode fait évoluer un simplexe dans l'espace de recherche en utilisant uniquement les valeurs de la fonction aux sommets. À chaque itération, les sommets sont évalués puis classés. On note B le meilleur sommet et W le pire :

$$f(B) \leq \cdots \leq f(W).$$

Le centre de gravité G est calculé avec tous les sommets sauf le pire :

$$G = \frac{1}{n} \sum_{i \neq W} X_i.$$

La première transformation est la réflexion :

$$R = G + \alpha(G - W), \qquad \alpha = 1.$$

Si la réflexion améliore fortement la solution, une expansion est essayée :

$$E = G + \gamma(R - G), \qquad \gamma = 2.$$

Lorsque la réflexion n'est pas suffisamment intéressante, l'implémentation utilise une contraction vers le pire sommet :

$$C = G + \rho(W - G), \qquad \rho = 0.5.$$

Si cette contraction échoue, le simplexe est réduit autour du meilleur sommet :

$$X_i' = B + \sigma(X_i - B), \qquad \sigma = 0.5.$$

La logique générale est donc :

$$\boxed{\text{Évaluer} \to \text{Trier} \to G \to \text{Réflexion} \to \text{Expansion / Réflexion / Contraction / Réduction}}.$$

Critère d'arrêt

Le programme utilise deux critères simultanément. Le premier mesure l'écart entre la valeur de la fonction au pire et au meilleur sommet :

$$\Delta f = f(W) - f(B).$$

Le second mesure la taille du simplexe, calculée comme la plus grande distance entre le meilleur sommet et les autres sommets. L'algorithme s'arrête lorsque

$$\boxed{\Delta f < \varepsilon \quad \text{et} \quad \text{taille du simplexe} < \varepsilon}.$$

Une limite $N_{\max}$ sur le nombre d'itérations évite également une exécution trop longue. Dans les tests : $N_{\max} = 1000$ et $\varepsilon = 10^{-6}$.

4 Une collection de simplexes

La classe `CollectionSimplexe` conserve les différents états du simplexe :

$$S_0, S_1, S_2, \ldots, S_k.$$

Les simplexes sont stockés par valeur dans un `vector<Simplexe>`. L'instruction `push_back(simplexe)` crée donc une copie indépendante à chaque ajout. Cet historique permet d'étudier la convergence et de produire les rendus graphiques.

5 Héritage, copie et affectation

Une classe abstraite `Fonction` définit l'interface commune, en particulier la méthode `evaluer(point)`. La classe `Polynome` en hérite. Grâce au polymorphisme, Nelder–Mead travaille avec une référence de type `const Fonction&` sans connaître le type concret de la fonction.

La classe `Simplexe` possède aussi un constructeur par copie et un opérateur d'affectation explicitement définis. Le `main` en montre l'utilisation avec une copie de `s2`, puis une affectation. Comme les coordonnées sont stockées dans des `std::vector`, aucune gestion manuelle de mémoire n'est nécessaire.

6 Critère d'arrêt et coût de calcul

Les paramètres $N_{\max}$ et ε permettent de contrôler le compromis entre durée du calcul et précision. Une amélioration possible serait de mémoriser les valeurs de la fonction associées aux sommets inchangés afin d'éviter certaines évaluations répétées à chaque itération.

7 Tests et rendus

Les tests utilisent des fonctions dont le minimum théorique est connu. Ils permettent de comparer le résultat numérique à la solution exacte et d'observer l'évolution du simplexe.

	Test 1D	**Test 2D**	**Test 3D**
Fonction	$(x-2)^2$	$(x-2)^2+(y-3)^2$	$(x-1)^2+(y-2)^2+(z-3)^2$
Minimum théorique	$x^* = 2$	$(x^*, y^*) = (2, 3)$	$(x^*, y^*, z^*) = (1, 2, 3)$
Valeur théorique	0	0	0
Minimum obtenu	$x \simeq 2$	$(x, y) \simeq (2, 3)$	$(x, y, z) \simeq (1, 2, 3)$
Valeur obtenue	$\approx 1.42 \times 10^{-14}$	$\approx 1.40 \times 10^{-13}$	très proche de 0

Les résultats sont cohérents avec les minima théoriques. Les faibles écarts à zéro proviennent de la précision numérique des calculs et du critère de convergence utilisé.

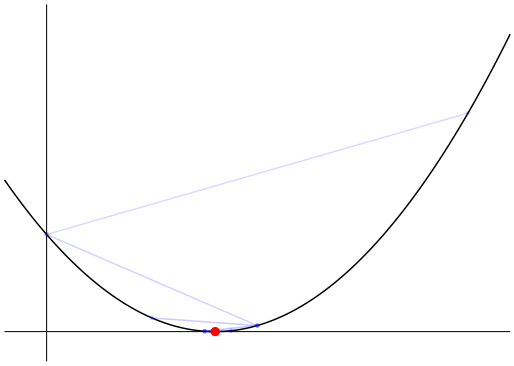


Figure 1 – Évolution du simplexe en dimension 1.
Le segment se déplace puis se contracte autour de $x = 2$.

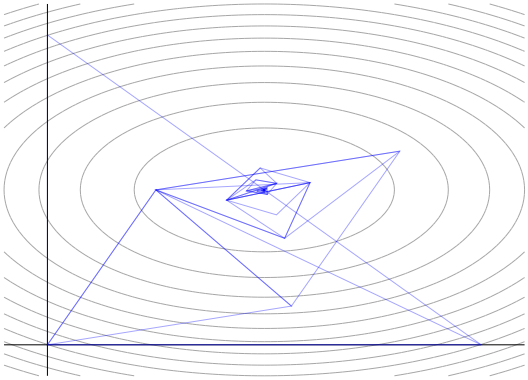


Figure 2 – Évolution des simplexes en dimension 2.
Les triangles successifs convergent vers le voisinage du point $(2, 3)$.

En dimension 1, le simplexe est un segment constitué de deux points. En dimension 2, il s'agit d'un triangle ; les courbes de niveau permettent de visualiser son déplacement vers le minimum. Le test 3D vérifie que l'implémentation reste valable lorsque le simplexe possède quatre sommets, même si aucun rendu 3D n'est produit.

8 Extension : recherche sous contrainte

L'extension du projet consiste à rechercher le minimum d'une fonction sur un hyperplan. Un hyperplan de $\mathbb{R}^n$ s'écrit

$$a_1x_1 + \cdots + a_nx_n = b.$$

La classe `FonctionSurHyperplan` choisit comme variable pivot celle dont le coefficient est le plus grand en valeur absolue, afin d'éviter une division par un coefficient très petit. Les $n-1$ autres coordonnées deviennent des paramètres libres. La coordonnée pivot est ensuite reconstruite à partir de l'équation de l'hyperplan. Le problème contraint en dimension n devient ainsi un problème non contraint en dimension $n-1$, ce qui permet de réutiliser directement la classe `NelderMead`.

Exemple utilisé dans le programme

On reprend

$$f(x, y) = 13 - 4x - 6y + x^2 + y^2 = (x-2)^2 + (y-3)^2$$

avec la contrainte

$$\boxed{x + y = 3}.$$

Les deux coefficients de la contrainte sont égaux. Dans l'implémentation actuelle, le premier coefficient maximal est choisi comme pivot : c'est donc x qui est reconstruit. On écrit

$$x = 3 - y.$$

En posant $t = y$, la fonction réduite devient

$$g(t) = f(3-t, t) = 2t^2 - 8t + 10 = 2(t-2)^2 + 2.$$

Le minimum du problème réduit est donc obtenu pour $t = 2$. On reconstruit alors

$$y = 2, \qquad x = 3 - 2 = 1,$$

d'où le minimum contraint

$$\boxed{(x, y) = (1, 2)}, \qquad \boxed{f(1, 2) = 2}.$$

La contrainte est bien vérifiée : $1 + 2 = 3$.

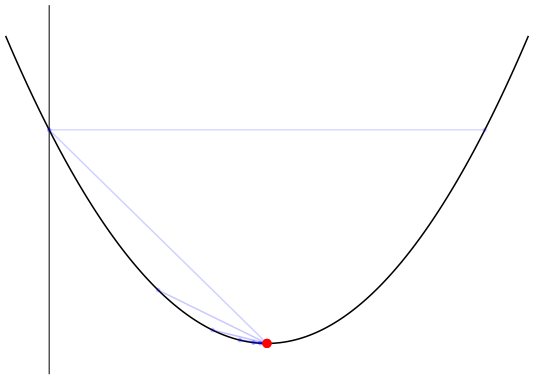


Figure 3 – Évolution de la recherche après réduction du problème contraint à une dimension.

Le minimum sans contrainte de la fonction est $(2, 3)$, mais ce point ne vérifie pas $x + y = 3$. L'algorithme cherche donc uniquement parmi les points autorisés par l'hyperplan et obtient $(1, 2)$. Cette extension illustre l'intérêt de l'héritage : `FonctionSurHyperplan` fournit une nouvelle fonction compatible avec la même interface `Fonction`, sans modifier l'algorithme de Nelder–Mead.